

Chương 1.NHẬP MÔN

CẤU TRÚC DỮ LIỆU và THUẬT TOÁN 1

Thầy giáo : PGS.TS.Hoàng Nghĩa Tý, đt 0913570291

(Chú ý : trong bài sử dụng font Times New Roman và .VnTime, nên nếu có chỗ nào lỗi font thì chọn(bôi đen) phần text đó rồi chọn lại font .VnTime)

1.1. Giới thiệu môn học

Thuật toán và cấu trúc dữ liệu đ-ợc coi là môn học cốt lõi của ngành Công nghệ thông tin. Niklaus Wirth - tác giả cuốn “ Cấu trúc dữ liệu +Giải thuật= Chương trình” đã phân tích tầm quan trọng của cấu trúc dữ liệu và thuật toán. Ch-ơng trình tốt cực là những mô tả cụ thể của các thuật toán trừu t-ợng dựa vào những biểu diễn và cấu trúc dữ liệu đặc biệt. Ch-ơng trình và dữ liệu không thể tách rời nhau đ-ợc. Dữ liệu đ-ợc xử lý bởi ch-ơng trình, cần đ-ợc tổ chức thành các cấu trúc sao cho nó phản ánh mối quan hệ giữa các mục dữ liệu và cho phép xử lý dữ liệu có hiệu quả.

Sinh viên của Khoa CNTT không thuộc chuyên ngành Công nghệ phần mềm, Kỹ thuật máy tính học môn Cấu trúc dữ liệu và thuật toán chỉ có 2 tín chỉ, tên học phần là *Kiến thức cơ bản về Cấu trúc dữ liệu và thuật toán*.

Sinh viên của Khoa CNTT thuộc chuyên ngành Công nghệ phần mềm, Kỹ thuật máy tính học môn Cấu trúc dữ liệu và thuật toán có 4 tín chỉ, chia làm 2 học phần, học phần đầu là *Kiến thức cơ bản về Cấu trúc dữ liệu và thuật toán*, học phần sau là *Cấu trúc dữ liệu và thuật toán nâng cao*.

Khi cần giải một bài toán bằng máy tính, người lập trình viên cần phải thực hiện lần lượt các công việc sau :

A) Tìm cách giải bài toán, gọi là tìm giải thuật hay thuật toán. Thuật toán có thể diễn đạt bằng lời, gạch đầu dòng hay vẽ thành sơ đồ. Một bài toán có thể có nhiều cách giải, khi đó cần chọn thuật toán nào tốt nhất. Tiêu chí tốt của thuật toán liên quan đến việc tổ chức dữ liệu, thời gian xử lý thuật toán, dễ dàng trong việc lập trình.

Các chuyên gia CNTT đã nghiên cứu, phát triển, hệ thống hóa các loại thuật toán đã có từ trước tới nay thành một lĩnh vực chuyên sâu, bất kỳ một lập trình viên chuyên nghiệp nào cũng phải nắm bắt được những thuật toán cơ bản, hay dùng trong thực tế.

Những thuật toán cơ bản thông dụng là :

- Các thuật toán sắp xếp (sắp xếp thường, sắp xếp nổi bọt, sắp xếp chèn, sắp xếp nhanh..);
- Các thuật toán tìm kiếm(tìm kiếm tuần tự, tìm kiếm nhị phân);
- Thuật toán đệ quy;
- Thuật toán truy hồi – quy hoạch động;
- ...

B) Tổ chức dữ liệu, viết chương trình, diễn đạt thuật toán thành các câu lệnh bằng một ngôn ngữ lập trình.

B1.Tổ chức dữ liệu

Khi tìm cách giải bài toán, xây dựng thuật toán, người lập trình đã phải hình dung dữ liệu được bố trí như thế nào và xử lý theo cách nào. Dữ liệu có thể là từng số, từng chữ đơn lẻ hay là một phần tử chứa nhiều dữ liệu thành viên? Các phần tử dữ liệu đứng riêng lẻ hay tạo thành dãy một chiều, một mảng 2,3 chiều v.v...?

Tùy theo ngôn ngữ lập trình nào được sử dụng mà các loại dữ liệu đó được cài đặt bằng những thành phần với các câu lệnh cụ thể.

Ngoài việc dữ liệu được tổ chức như thế nào, còn phải xem việc dữ liệu nhập vào -> xử lý -> đưa ra theo trật tự nào để chọn cấu trúc dữ liệu cho phù hợp.

Thông thường có 3 dạng trong quy trình **Dữ liệu vào → Xử lý → Kết quả ra** :

- ✓ Dạng tuần tự : nhập hết dữ liệu -> Xử lý từ đầu đến cuối -> Đưa ra từ đầu đến hết dữ liệu
- ✓ Vào trước – xử lý trước- Ra trước (cấu trúc dữ liệu kiểu xếp hàng – Queue)
- ✓ Vào sau, xử lý trước và ra trước (cấu trúc dữ liệu kiểu ngăn xếp – stack)

B2) Lập trình

Sau khi đã có thuật toán, đã xác định cách tổ chức dữ liệu, sẽ tiến hành lập trình, diễn đạt thuật toán thành các câu lệnh

Trong quá trình lập trình cần chạy thử (biên dịch và chạy – compile and run) và tìm lỗi.

Thường có 2 loại lỗi chính :

- Lỗi cú pháp: theo thông báo lỗi ở phần dưới màn hình để sửa lỗi. Mỗi lần sửa xong biên dịch và chạy thử lại cho đến khi hết lỗi.
- Lỗi thuật toán hay lỗi công thức, phép tính, cách tính: Chương trình chạy được nhưng kết quả sai là do lỗi thuật toán, cần phải kiểm tra, sửa lại phép tính, cách tính hay cả thuật toán.

1.2. Các khái niệm

1.2.1. Dữ liệu ở dạng mã máy và ý nghĩa của chúng

Trong máy tính các giá trị dữ liệu được đọc l-u trữ dưới dạng các bit, là các số ở hệ đếm nhị phân (0 và 1). Các bit được tổ chức thành các nhóm gọi là các Từ máy (word), tức là mỗi word chứa một số cố định các bit. Độ dài của word thay đổi theo loại máy tính (máy 16 bit hay 32 bit, 64 bit). Các Từ máy này được đánh địa chỉ bắt đầu từ 0, vì vậy có thể truy cập vào một word bất kỳ theo địa chỉ của nó.

Khi ở dạng một dãy bit, nó có thể biểu diễn nhiều ý nghĩa khác nhau.

Trong từng ngôn ngữ lập trình cụ thể, ngôn ngữ lập trình sẽ biểu diễn các dãy bits này dưới dạng các cấu trúc xác định, cho phép xử lý và diễn đạt được các dữ liệu.

Những khái niệm quan trọng liên quan đến dữ liệu là Kiểu dữ liệu, Cấu trúc dữ liệu, Cấu trúc dữ liệu tĩnh, Cấu trúc dữ liệu động, .

1.2.2. Kiểu dữ liệu

Các biến tham gia trong chương trình thường có giá trị kiểu số nguyên hay số thực, 1 ký tự chữ hay một chuỗi ký tự.

Ứng với mỗi kiểu, giá trị của biến sẽ chiếm một số lượng byte bộ nhớ tương ứng. Tất cả những thông tin này phải được khai báo trước khi có lệnh thực hiện trong chương trình

Kiểu ký tự được khai báo bởi từ khóa char (viết tắt chữ character — ký tự), kiểu số nguyên – int(integer), kiểu số thực – float(floating – số thập phân), số thực có độ chính xác gấp đôi – double .

Bảng về kiểu dữ liệu, dung lượng bộ nhớ chiếm dụng để lưu giá trị, giá trị lớn nhất và nhỏ nhất có thể được lưu giữ với các kiểu biến đó:

Kiểu	Độ rộng bit	Dãy giá trị
char	1 byte	-127 tới 127 hoặc 0 tới 255
int	4 byte	-2147483648 tới 2147483647
short int	2 byte	-32768 tới 32767
long int	4 byte	-2,147,483,647 tới 2,147,483,647
float	4 byte	+/- 3.4e +/- 38 (~7 chữ số)
double	8 byte	+/- 1.7e +/- 308 (~15 chữ số)

(Chú ý : *tên kiểu viết thường, không viết hoa*)

Biến kiểu ký tự **char** chiếm **1 byte** bộ nhớ, giá trị mã từ -128 đến 127.

Sau khi được khai báo, biến kiểu **char** có thể được nhận giá trị theo 2 cách tương đương – **chữ** hoặc **giá trị số**.

Trong bảng mã ASCII, mỗi cột có bên trái là số từ 0 đến 255, bên phải là ký tự ứng với số mã ở bên trái, ví dụ mã 65 là chữ A,..., 97 là chữ a, 62 là >, 63 là ?, v,v...

Ví dụ 1 : s là một biến ký tự thì câu lệnh gán s = 'A' cũng tương đương với câu lệnh gán

s = 65.

Khái niệm *kiểu dữ liệu* rất quan trọng, nó xác định tập hợp giá trị mà biến có thể nhận. Mô tả kiểu sẽ quy định loại giá trị của biến và cung cấp cho chương trình dịch những thông tin cần thiết.

Kiểu dữ liệu còn quy định những phép toán có thể thực hiện được đối với các dữ liệu.

a) Dữ liệu kiểu nguyên - int

Dữ liệu kiểu **int** được lưu trữ trong các *từ máy*. Tùy theo loại máy tính, độ dài của *từ máy* giới hạn phạm vi của các số nguyên lưu trữ được. Số nguyên lớn nhất lưu trữ được trong một *từ máy* 8 bit là $2^7-1=127$, trong *từ máy* 16 bit là $2^{15}-1=32767$, trong một *từ máy* 32 bit là $2^{31}-1=2.147.483.647$.

b) Dữ liệu kiểu số thập phân - float

Trong hệ đếm nhị phân, một số thực bất kỳ có thể biểu diễn bằng tổng các số với mũ của 2

$$X = \sum_{i=-m}^{i=n} a_i \cdot 2^i \quad \text{trong đó } a_i = 0 \text{ hoặc } 1$$

Có hai cách lưu số thực : dấu chấm tĩnh (fixed point) và dấu chấm động (floating point). Ví dụ 110.101 (6.625) hay 0.110101×2^3

Một phần của *từ máy* được dùng để lưu một số cố định các bit của phần định trị, phần khác lưu phần mũ.

<Phần định trị>.<phần mũ>

c) Dữ liệu kiểu ký tự - char

Máy 1-u trữ dữ liệu ký tự dựa trên cơ sở gán mã số cho mỗi ký tự. Các ký tự đ-ợc biểu diễn bằng mã nhị phân, mỗi từ máy 16 bit chia thành hai byte, mỗi byte 1-u trữ một ký tự d-ới dạng nhị phân. Ký tự đ-ợc mã hoá theo ASCII (American Standard Code for information Interchange — Bộ m-ó chuẩn của Mỹ dựng để trao đổi thụng tin) hay EBCDIC (extended Binary Code Decimal Interchange Code- Mã BCD mở rộng). Phép toán phổ biến cho các ký tự là so lựa tuần tự (collating sequence).

Ví dụ : ký tự A trong bảng mã ASCII có mã là 65, ký tự B trong bảng mã ASCII có mã là 66.

Khi đó phép so sánh $A > B$ sẽ cho kết quả sai.

d) Dữ liệu kiểu chuỗi(xâu) ký tự - string

d1.Khai báo hàm tiêu đề : `#include <string>`

d2.Khai báo kiểu string : `string <tên biến>;`

ví dụ :

```
string xau;
```

```
string xau1="Cau truc du lieu va Thuat toan";
```

d3.Các phép toán đối với chuỗi(xâu) ký tự

➤ Gán chuỗi =, ví dụ

○ `string xau1="Cau truc du lieu va Thuat toan";`

➤ Các phép nối chuỗi : `+, +=`

➤ Các phép so sánh 2 chuỗi : `==, !=, >, >=, <, <=`

Hoặc dùng hàm so sánh 2 chuỗi `compare()`

➤ Một số hàm về chuỗi :

✓ Hàm cho độ dài chuỗi : `length()`

✓ Trích lấy chuỗi con trong chuỗi đã cho : `substr(<vị trí>, <số ký tự>)`

Ví dụ : `string y, x="12345";`

```
y=x.substr(2,3);
```

```
cout<<y; // kết quả cho 234
```

✓ Chèn một chuỗi s1 vào giữa chuỗi s khác : `s1.insert(int vitri, string s);`

Chương trình con về chuỗi string :

```
#include <iostream>
```

```
#include <conio.h>
```

```
#include <string>
```

```
using namespace std;
```

```
int main()
```

```
{ cout << " Các phép toán với string " << endl;
```

```
string xau,xau1="Giao Trinh";
```

```
string xau2="Cau truc du lieu va Thuat toan";
```

```

xau=xau1+xau2;
cout<<"  Do dai xau = "<<xau.length()<<"\n";
if (xau1.compare(xau2) != 0)
cout << " Chuoi ' '"<<xau1 << " ' khac voi chuoi ' '" << xau2 << " '\n";
system("pause");
return 0;
}

```

d4.Nhập dữ liệu cho biến kiểu string và biến mảng ký tự

Trường hợp 1. Nhập dữ liệu cho **biến kiểu string** , trong đó có thể có dấu cách (khoảng trắng)

Trong trường hợp này, dùng các lệnh sau :

```

cin.ignore(); //loại bỏ dấu trống ở đầu
getline(cin, Tên_mảng[i].Tên-Thành-Phần);
//Nhập vào các thành phần của phần tử thứ i của mảng
Ví dụ : getline(cin,dslop[i].hoten);
fflush(stdin); //xóa bộ nhớ đệm

```

Trường hợp 2. Nhập cho biến kiểu mảng ký tự .

Dùng các lệnh

```

cin.ignore(); // loại bỏ dấu trống ở đầu
cin.get(<tên mảng>[i].<tên thành phần>,<số ký tự>);
Ví dụ      cin.ignore();cin.get(lop[i].hoten,30);

```

e)Dữ liệu kiểu Logic

Mỗi giá trị logic đ- ọc biểu diễn bởi một chữ số ở hệ đếm nhị phân là 0 hoặc 1. Trong các ngôn ngữ thông dụng có 3 phép toán logic là &&(AND), ||(OR),!(NOT).

Cho biến A có giá trị 1 và biến B có giá trị 0, thì khi đó:

Toán tử	Miêu tả	Ví dụ
&&	Gọi là toán tử Logic AND (Và), chỉ bằng 1(true) khi cả hai toán hạng đều có giá trị khác 0.	(A && B) là false.
 	Gọi là toán tử Logic OR (Hoặc). chỉ bằng 0(false) khi cả hai toán hạng đều có giá trị bằng 0.	(A B) là true.
!	Gọi là toán tử Logic NOT (Phủ định), cho kết quả đảo ngược trạng thái logic của toán hạng đó.	!(A && B) là true.

1.2.3.Cấu trúc dữ liệu - Data Structure

Khái niệm : Cấu trúc dữ liệu là cách thức tổ chức l-u trữ dữ liệu trong bộ nhớ máy tính.

Trong các loại cấu trúc dữ liệu, ng-ời ta phân chia ra cấu trúc đơn giản, cấu trúc phức tạp, cấu trúc tĩnh, cấu trúc động.

Cấu trúc dữ liệu đơn giản là các kiểu dữ liệu nguyên thuỷ đ-ợc định nghĩa trong các ngôn ngữ lập trình, ví dụ nh- int, float, char trong ngôn ngữ C++.

Trong thực tế , có những bài toán mà cách tổ chức dữ liệu đơn giản không đáp ứng đ-ợc, đòi hỏi phải tổ chức phức tạp hơn, ví dụ nh- một dãy số, một danh sách các dữ liệu,v.v...

Cấu trúc dữ liệu tĩnh : là cách tổ chức mà kích th-ớc của các phần tử dữ liệu là cố định, cấu trúc cũng cố định trong khi ch-ơng trình dịch làm việc. Trong ngôn ngữ C++, các kiểu mảng(dãy), kiểu dữ liệu liệt kê, hay các kiểu nguyên thuỷ nh- int , float,...

Cấu trúc dữ liệu động là cách tổ chức mà có thể thay đổi cả kích th-ớc và cấu trúc của các dữ liệu. Các dữ liệu động này đ-ợc phát sinh ra trong quá trình ch-ơng trình thực hiện. Chúng không đ-ợc thể hiện khi dịch ch-ơng trình, và th-ờng đ-ợc sử dụng con trỏ để tạo ra dữ liệu động

Ngoài các khái niệm trên, để phục vụ cho việc học tập , nghiên cứu và triển khai các ứng dụng, ng-ời ta còn chia ra Cấu trúc dữ liệu tuyến tính, cấu trúc dữ liệu phi tuyến. Những cấu trúc mà khi xử lý đ-ợc tiến hành ở dạng một dãy liên tục các dữ liệu thì đ-ợc nhóm gộp vào loại tuyến tính, ví dụ nh- mảng, stack,queue, list đơn giản. Những cấu trúc còn lại đ-ợc gọi chung là phi tuyến.

1.3.Các mô hình dữ liệu

Để đảm bảo hiệu quả các quá trình xử lý thông tin cần phải tổ chức dữ liệu cho phù hợp. Mô hình hoá dữ liệu phải phản ánh đ-ợc thế giới hiện thực một cách tốt nhất và phải cài đặt đ-ợc trong máy tính.

Từ Data(dữ liệu) có xuất xứ từ thuật ngữ “datum” tiếng Hy Lạp có nghĩa là “sự kiện”. Tuy nhiên không phải bao giờ dữ liệu cũng t-ơng ứng với một sự kiện cụ thể nào đó, hiện diện trong thế giới thực. Chúng ta gọi dữ liệu là mô tả của một hiện t-ợng bất kỳ (hay là một ý t-ởng) mà đáng giá để biểu diễn nó và xác định nó chính xác. Từ xa xưa con ng-ời đã sử dụng nhiều loại ph-ơng tiện khác nhau để ghi nhận các sự kiện, ý t-ởng: đá, giấy,... thông th-ờng dữ liệu (các sự kiện - Data) và diễn giải nó (ngữ nghĩa - semantic) đ-ợc ghi cùng nhau vì ngôn ngữ tự nhiên khá tinh tế, phong phú để biểu diễn hiện t-ợng , sự kiện. Khi ta nói " dung l-ợng đĩa cứng 256 GB" thì ở đây 256 là dữ liệu, còn ngữ nghĩa là “dung lượng GB”. Trong một số trường hợp thì dữ liệu và ngữ nghĩa (semantic) bị tách rời (ví dụ: bảng giờ tàu — diễn giải ở bên trên, còn phía d-ới là giờ chạy của các tàu ở các tuyến đ-ờng khác nhau).

Trong công nghệ thông tin thì dữ liệu và ngữ nghĩa càng bị phân tách. Trong bộ nhớ của MTĐT, ng-ời ta chỉ làm việc với dữ liệu, không để ý đến ngữ nghĩa, phân ngữ nghĩa của dữ liệu bản thân ng-ời lập trình phải ngầm hiểu, l-u giữ ở bộ nhớ riêng. Th-ờng trong ch-ơng trình phải ghi chú về ý nghĩa của các loại dữ liệu, thiếu ghi chú này thì dữ liệu chỉ là các dãy bit đơn thuần.

Sự linh hoạt của biểu diễn dữ liệu đạt đ-ợc nhờ hai ph-ơng pháp:

- Có nhiều cách nhìn nhận khác nhau đối với cùng một đối t-ợng dữ liệu
- Biểu diễn đồng nhất hoá các dữ liệu khác nhau.

Ví dụ : với đối t-ợng là Con ng-ời , ta có hai cách tiếp cận :

Theo ph-ong pháp thứ nhất : cùng một đối t-ợng con ng-ời , nh-ng trong danh sách lớp học thì đó là họ tên sinh viên, trong bài toán xét lên l-ợng lại là cán bộ công nhân viên (CBCNV), trong quản lý bệnh viện đó là họ tên bệnh nhân. ..

Theo ph-ong pháp thứ hai, ta có thể coi mọi ng-ời từ Giám đốc, tr-ởng phó phòng, đến nhân viên ... đều là CBCNV..

Đó là những lý do dẫn đến cần phải trừu t-ợng hoá dữ liệu. Ph-ong tiện để biểu diễn dữ liệu gọi là Các mô hình dữ liệu(Data model). Mô hình dữ liệu là ph-ong tiện để trừu t-ợng hoá dữ liệu, cho khả năng nhìn thấy “rừng” (nội dung thông tin của dữ liệu) chứ không chỉ “từng cây riêng rẽ” (các giá trị riêng lẻ của dữ liệu). Mô hình dữ liệu cho khả năng biểu diễn một phần ngữ nghĩa (semantic) của dữ liệu. Mô hình dữ liệu xác định các quy tắc mà theo đó sẽ cấu trúc hoá dữ liệu.

Tập hợp các dữ liệu mà có cấu trúc t-ợng ứng với một sơ đồ dữ liệu nhất định thì gọi là một cơ sở dữ liệu- Database.

Những mô hình dữ liệu cơ bản đã đ-ợc nghiên cứu nhiều là :

- mô hình phân cấp(thứ bậc — Hierarchical Model)
- mô hình mạng l-ới (Network Model)
- mô hình quan hệ (Relational Model)
- mô hình đối t-ợng/quan hệ (Object/Relational Model)
- mô hình h-ớng đối t-ợng (Object-Oriented Model)

D-ới đây ta sẽ xem xét một số mô hình , thứ tự đ-ợc trình bày theo mức độ phổ dụng.

1.3.1.Mô hình quan hệ - Relational Model

Mô hình dữ liệu quan hệ do Edgar F. Codd đề x-ớng trong những năm 1960.

Ph-ong tiện để cấu trúc hoá dữ liệu trong mô hình quan hệ là *Quan hệ* (Relation). Các quan hệ đ-ợc hiểu theo ý nghĩa của toán tập hợp và đ-ợc thể hiện d-ới dạng các bảng. Mô hình dữ liệu dựa trên các quan hệ, đ-ợc biểu diễn bằng các bảng lần đầu tiên đ-ợc Codd E.F. đề x-ớng trong tài liệu “A relational model of Data for large shared Data banks ".Commun.ACM,13,p.377-387.

Một quan hệ đ-ợc định nghĩa nh- sau :

Cho các tập hợp $D_1, D_2, \dots, D_n$ (không nhất thiết phải khác nhau), khi đó R là một quan hệ, đ-ợc cho trên các tập hợp này, nếu R là một tập hợp các corteges n -địa ph-ong hay đơn giản là tập hợp các corteges mà trong mỗi cortege đó phần tử thứ nhất thuộc D_1 , phần tử thứ hai thuộc $D_2, \dots$. Tập hợp D_i gọi là các Domen của R . Số n đ-ợc gọi là bậc của R , số l-ợng corteges là lực l-ợng của R .

Mô hình dữ liệu quan hệ là dạng mô hình bảng - mỗi bảng là một quan hệ. Mở rộng cơ sở dữ liệu quan hệ là tập hợp các bảng. Các cột của bảng gọi là các thuộc tính, mỗi hàng của bảng t-ợng ứng với một cortege của quan hệ. Để quản lý một cơ sở dữ liệu cần xây dựng một hệ quản trị . Trong các hệ quản trị cơ sở dữ liệu quan hệ, các thuộc tính còn đ-ợc gọi là các tr-ờng - field; các hàng là các cấu trúc - record .

Ví dụ. có 5 bảng

D1: hồ sơ thí sinh , D2: báo danh-phách, D3: phach1-diem1, D4: phach2-diem2, D5: phach3-diem3, là các Domen.

Hồ sơ thí sinh D1			
Số BD	Họ tên	Ngày sinh	Đối tượng
10265	P.T.Anh		

báo danh-phách D2			
Số BD	Phách1	Phách2	Phách3
10265	45	2050	5000

D3Phách1-diem1	
Phách1	Toán
45	7

D4 phách2-diem2	
Phách2	Điểm 2
2050	8

D5 phách3-diem3	
Phách3	Điểm 3
5000	8.5

Mỗi thí sinh có 1 số báo danh duy nhất.

Mỗi số báo danh chỉ có duy nhất 1 phách môn1, 1 phách môn 2, 1 phách môn3. Cả 3 số phách của 1 số báo danh không đ-ợc trùng nhau. Mỗi phách 1 có 1 điểm môn1, mỗi phách 2 có 1 điểm môn 2, mỗi phách 3 có 1 điểm môn 3 .

Từ các bảng này ta sẽ kết nối và truy xuất ra đ-ợc kết quả thi tuyển của từng thí sinh, đảm bảo không nhầm lẫn.

Các thao tác cơ bản đối với mô hình dữ liệu quan hệ là :

- Trích dữ liệu từ quan hệ để tạo một quan hệ mới theo điều kiện
- Cập nhật thông tin ; - Chèn, xóa các tr-ờng; - Chèn, xóa các bản ghi
- Sắp xếp các tr-ờng theo một trật tự nào đó
- Tìm kiếm thông tin theo các điều kiện

Các hệ cơ sở dữ liệu đ-ợc phổ cập phần lớn theo mô hình quan hệ, nổi bật là FOXPRO, ACCESS, . . .Các hệ quản trị đi kèm theo là Visual Foxpro, Visual Basic, ngôn ngữ SQL ...

1.3.2.Mô hình mạng lưới

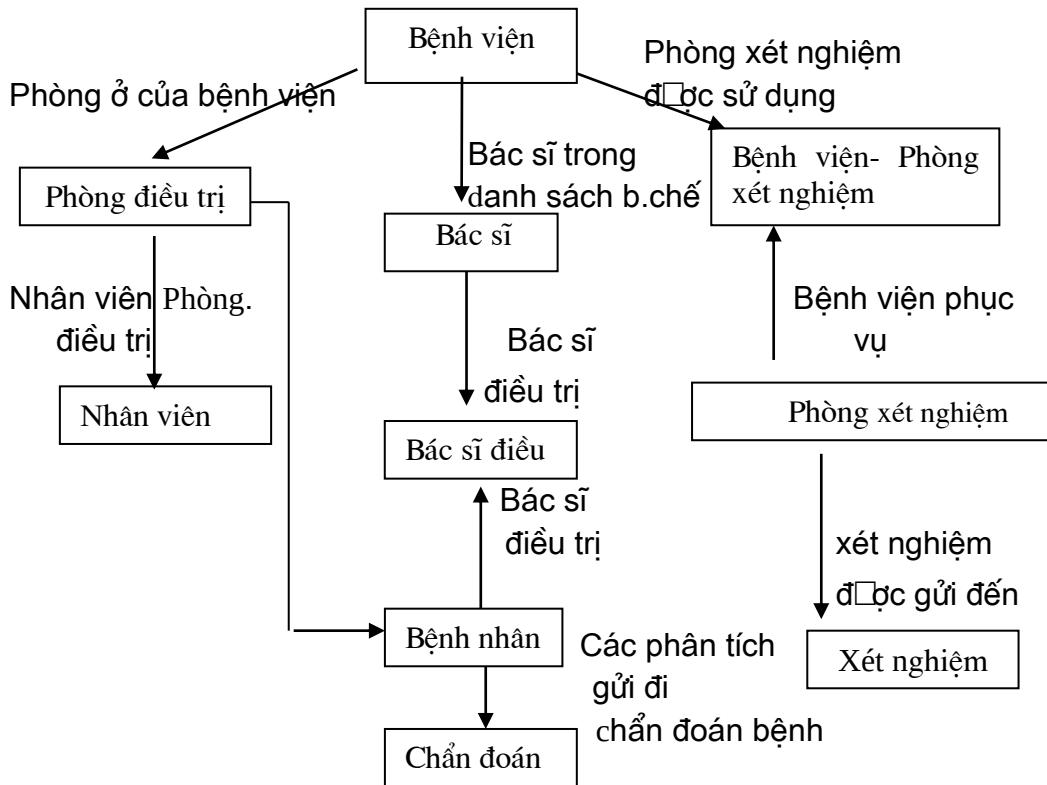
Trong thực tế, có nhiều bài toán có thể mô phỏng d-ới dạng một mô hình có nhiều mối quan hệ ngang dọc kiểu mạng l-ới. Năm 1971, Hội nghị về các ngôn ngữ Hệ thống Dữ liệu (the Conference on Data Systems Languages- CONDASYL) đã định nghĩa mô hình mạng l-ới. Cơ sở để mô hình hóa mạng l-ới là cấu trúc tập hợp. Một tập hợp bao gồm 1 kiểu cấu trúc chủ, 1 tên tập hợp, và 1 kiểu cấu trúc(struct) thành viên. Kiểu cấu trúc thành viên (mức con) có thể tham gia trong nhiều tập hợp, nghĩa là cho phép có nhiều cấu trúc ở mức trên (mức cha). Đến l-ợt mình, cấu trúc chủ cũng có thể là cấu trúc thành viên hay cấu trúc con trong các tập hợp khác.Mô hình dữ liệu kiểu mạng CONDASYL dựa trên lý thuyết tập hợp của Toán học.

Hai khái niệm quan trọng ở đây là cấu trúc(struct) và mối liên hệ.

Các kiểu cấu trúc dùng để biểu diễn bảng các kiểu đối tượng.

Ví dụ, để xây dựng phần mềm quản lý hệ thống các bệnh viện, ta dùng mô hình mạng l-ới để mô tả .

Một sơ đồ dữ liệu theo mô hình mạng lưới có dạng nh- sau:



Mỗi cấu trúc có các tr- ờng dữ liệu riêng :

Bệnh viện (mã bệnh viện, chức năng điều trị, địa chỉ, điện thoại, số l- ợng gi- ờng)

Phòng điều trị (mã phòng, chức năng điều trị, số l- ợng gi- ờng)

Ng- ời làm việc (số hiệu, họ tên, chức vụ, ca trực, mức l- ợng)

Bác sĩ (số hiệu, số đăng ký)

Bệnh nhân (số đăng ký, số gi- ờng, họ tên, địa chỉ, ngày sinh, nam/nữ, tên bệnh điều trị)

Chẩn đoán (mã chẩn đoán, kiểu chẩn đoán, mức trầm trọng, phòng ngừa)

Bệnh viện- phòng xét nghiệm (mã bệnh viện, số liệu phòng xét nghiệm)

Phòng xét nghiệm (số hiệu phòng xét nghiệm, tên gọi, địa chỉ, điện thoại)

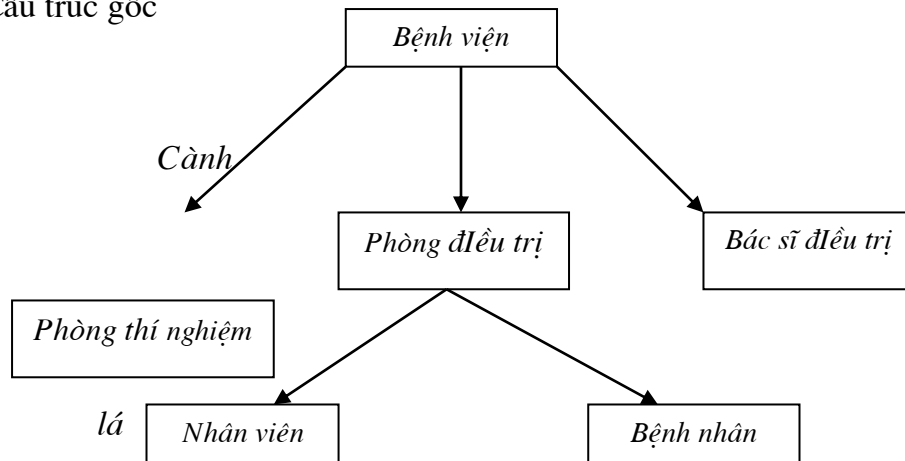
Xét nghiệm (mã xét nghiệm, kiểu, ngày ấn định, giờ ấn định, số ph- ơng án, tình trạng)

Trong sơ đồ dữ liệu kiểu mạng, có cấu trúc chủ và cấu trúc phụ thuộc (thành viên) ví dụ: *Bệnh viện* là cấu trúc chủ và *Phòng điều trị* — phụ thuộc . Hệ quản trị dữ liệu dạng mạng đ- ợc cài đặt nổi tiếng là Condasyll Data Base Task Group (DBTG) (1971)

1.3.3. Mô hình phân cấp – thứ bậc (Hierarchical model)

Trong mô hình phân cấp dữ liệu đ- ợc tổ chức dạng cây, cũng là một dạng của mô hình có kiểu đồ thị với các đỉnh là các bảng. Hệ quản trị dữ liệu dạng phân cấp nổi tiếng nhất là họ IMS (Information Management System/Virtual Storage) khi xây dựng dự án đổ bộ lên mặt trăng (Apollo) của Mỹ trong những năm 1960- 1970. Trong sơ đồ phân cấp, toán đồ cấu trúc là một cây đ- ợc sắp trật tự.

-Cấu trúc gốc



Bệnh viện (m□ b□nh vi□h, t□h,ăă ch□ăi□h tho□, s□gi□□hng)

Phòng xét nghiệm (s□hi□u ph□ng x□t nghi□m,t□h g□, ăă ch□tel)

Phòng điều trị(m□ ph□ng, t□h g□, s□gi□□hng)

Nhân viên (s□hi□u nhân vi□h, h□t□h, ch□c v□)

Bệnh nhân(s□ă□ng k□s□gi□□hng,h□t□h,ăă ch□ngày sinh,nam/nệ, b□nh y□n)

Bác sỹ(s□hi□u bỹc s□ h□t□h, chuy□n m□h)

Trong sơ đồ này có cấu trúc cha, cấu trúc con. Dữ liệu là 1 dãy các bản ghi, mỗi cấu trúc chứa các tr-ởng giá trị. Mô hình dữ liệu sẽ tập hợp tất cả các thành phần dữ liệu thành các cấu trúc . Các kiểu cấu trúc này t-ởng đ-ởng các bảng trong mô hình dữ liệu quan hệ, mỗi cấu trúc riêng rẽ t-ởng đ-ởng 1 dòng trong bảng.Để tạo các quan hệ giữa các kiểu cấu trúc, Mô hình thứ bậc dùng Quan hệ — Cha Con.

1.3.4.Mô hình Đối t-ởng/Quan hệ

Đây là một sự mở rộng của các Hệ thống quản trị dữ liệu dạng đối t-ởng- quan hệ (ORDBMSs), chúng cho phép l-u trữ thêm đối t-ởng mới vào hệ thống quan hệ trong trung tâm của các hệ thống thông tin hiện đại. Những tiện ích mới này đã cho phép tích hợp việc quản lý các cơ sở dữ liệu truyền thống, các đối t-ởng phức tạp nh- các dãy thời gian, các dữ liệu về vũ trụ, các dữ liệu đa ph-ởng tiện nh- âm thanh, hình ảnh, phim, các applets(là những ch-ởng trình viết bằng ngôn ngữ Java mà có thể nhúng vào trang HML). Bằng các ph-ởng pháp tổ hợp, đóng gói với các cấu trúc dữ liệu, một máy chủ ORDBMS có thể thực hiện các thao tác xử lý dữ liệu phức tạp đối với các dữ liệu đa ph-ởng tiện hay các đối t-ởng phức tạp khác.

Mô hình đối t-ởng/quan hệ đã tích hợp các -u việt của mô hình dữ liệu dạng quan hệ và tính mềm dẻo của mô hình định h-ởng đối t-ởng. Kỹ s- thiết kế cơ sở dữ liệu có thể làm việc với cấu trúc dạng bảng dữ liệu quen thuộc và với ngôn ngữ định nghĩa dữ liệu DDL(Data Definition Language) khi xử lý các khả năng quản lý đối t-ởng mới.

Những ngôn ngữ xử lý ORDBMS là các ngôn ngữ quen thuộc nh- SQL,các ODBC (Open Data Base Connectivity),JDBC(Java Data Base Connectivity) .

1.3.5.Mô hình định h-ởng đối t-ởng

Hệ Cơ sở dữ liệu định h-ởng đối t-ởng OODB(Object-Oriented Data Base) là tổ hợp của hệ thống ngôn ngữ lập trình h-ởng đối t-ởng và hệ thống l-u trữ dữ liệu. Sức mạnh của OODB có

đ- ọc nhờ việc đ- ỡ phải xử lý dữ liệu đang l- u trữ cũng nh- dữ liệu đ- ọc truyền đến hay dữ liệu đang xử lý trong các ch- ơng trình.

Trong Hệ quản trị Cơ sở dữ liệu quan hệ, các cấu trúc dữ liệu phức tạp đ- ọc trải ra thành các bảng dữ liệu hoặc kết hợp các bảng lại theo cấu trúc bộ nhớ, ng- ọc lại, trong hệ quản trị dữ liệu h- ớng đối t- ợng không thực hiện việc l- u trữ hay khôi phục các đối t- ợng liên kết trong của trang web hay trong mô hình thứ bậc. Với sự sắp xếp 1-1 giữa các đối t- ợng của ngôn ngữ lập trình h- ớng đối t- ợng với các đối t- ợng của cơ sở dữ liệu sẽ có 2 lợi ích hơn so với các cách l- u trữ khác: nó cho phép thực hiện việc quản lý các đối t- ợng hoàn hảo hơn, đồng thời nó cho phép quản lý tốt hơn các quan hệ phức tạp giữa các đối t- ợng. Những tính - u việt này của hệ quản trị cơ sở dữ liệu h- ớng đối t- ợng đã làm cho nó hỗ trợ tích cực hơn cho các phần mềm ứng dụng nh- phần mềm phân tích rủi ro tài chính của công ty, phần mềm dịch vụ viễn thông, cấu trúc tài liệu WEB, hệ thống tự động hoá thiết kế và sản xuất,v,v...

1.2.6.Mô hình nửa cấu trúc

Trong mô hình dữ liệu nửa cấu trúc, thông tin th- ờng đ- ọc liên kết với một sơ đồ mà nó chỉ tường minh thông qua bản thân dữ liệu, đôi khi còn gọi là “tự mô tả”. Trong cơ sở dữ liệu loại này, không có sự phân biệt rõ ràng giữa dữ liệu và sơ đồ, mức độ cấu trúc của dữ liệu phụ thuộc vào phần mềm ứng dụng.

1.2.7.Mô hình dữ liệu liên kết

Mô hình dữ liệu liên kết chia các vật thể của thế giới thực mà dữ liệu của chúng đ- ọc ghi d- ới 2 dạng : các thực thể - đó là các đối t- ợng tồn tại một cách rời rạc, độc lập; các mối liên kết — là những thứ tồn tại phụ thuộc vào một hay nhiều vật thể khác.

Ngoài các môn hình dữ liệu trên, hiện có thêm 2 mô hình mới là *Mô hình Thực thể - Thuộc tính - Giá trị (EAV)* và *mô hình dữ liệu ngữ cảnh - Context Model*).

1.4. Ôn tập về kiểu cấu trúc, mảng và con trỏ, lớp-class

1.4.1.Cấu trúc – structure

Trong nhiều bài toán quản lý, các đối tượng quản lý có các thành phần thông tin thuộc các kiểu dữ liệu khác nhau .

Ví dụ : khi quản lý sinh viên, mỗi sinh viên có các thông tin :

- Mã sv : gồm cả chữ cả số, nó thuộc kiểu **mảng char[]** hay **string**- chuỗi
- Họ tên : thuộc kiểu chuỗi **string**, cũng có thể kiểu **mảng char[]**
- Ngày sinh : kiểu số nguyên **int**;
- Điểm các môn, điểm trung bình : **float**

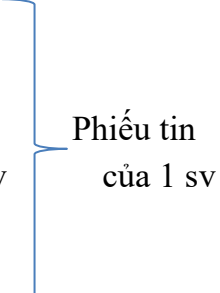
Trước đây, để lưu mã sinh viên, họ tên, ngày sinh, điểm môn học của từng người đã dùng các dãy (mảng 1 chiều) `masv[n]`, `hoten[n]`, `ngaysinh[n]`, `diem[n][m]`... Cách làm này đơn giản nhưng hơi rườm rà. Trong C++ có kiểu dữ liệu **struct – cấu trúc**, cho phép chỉ dùng 1 dãy (1 mảng 1 chiều) để lưu thông tin cả lớp, mỗi phần tử của dãy sẽ là một **struct** lưu hết các thành phần thông tin của từng người. Lúc đó, thông tin của cả lớp chỉ là 1 dãy(mảng) có n phần tử , mỗi phần tử có kiểu **struct** .

a)Cú pháp khai báo kiểu cấu trúc

```
struct <tên kiểu cấu trúc>// <tên kiểu cấu trúc> do ta tự đặt  
{ <kiểu dữ liệu> <tên trường dữ liệu thành phần>;  
.....  
<kiểu dữ liệu> <tên trường dữ liệu thành phần >;  
};
```

Ví dụ . với bài toán quản lý sinh viên nói trên, có thể khai báo kiểu cấu trúc sinh viên như sau :

```
struct sinhvien //kiểu cấu trúc  
{  
    char masv[10];  
    string hoten;//Le Hong Hanh  
    int ngaysinh;//05092001  
    float diem[15];//điểm của 15 môn của 1 sv  
    float diemtb;  
};
```



b)Khai báo biến cấu trúc

Sau khi khai báo **kiểu cấu trúc**, để sử dụng được kiểu cấu trúc đó phải khai báo **biến cấu trúc**, là biến có kiểu cấu trúc đã được định nghĩa trước đó.

Có 2 cách khai báo biến cấu trúc.

Cách 1: tên biến cấu trúc được viết ngay sau dấu } kết thúc định nghĩa kiểu cấu trúc .

Ví dụ. đặt tên biến sv có kiểu cấu trúc sinhvien :

```
struct sinhvien  
{ char masv[10];  
    string hoten;  
    int ngaysinh;  
    float diem[10];  
    float diemtb;  
}sv;
```

Cách 2. tên biến kiểu cấu trúc được viết ở dòng riêng biệt theo cú pháp :

```
struct <kiểu-cấu-trúc> < tên-biến-cấu trúc>;
```

Ví dụ.

```
struct sinhvien  
{ char masv[10];  
    string hoten;  
    int ngaysinh;  
    float diem[10];  
    float diemtb;  
};  
struct sinhvien sv; //sv là biến cấu trúc có kiểu cấu trúc sinhvien
```

Chương trình chỉ làm việc với **tên biến cấu trúc**, tên kiểu cấu trúc chỉ để xác định **kiểu dữ liệu cấu trúc** của biến.

c) Truy cập biến cấu trúc

Để truy nhập đến từng thành phần của cấu trúc ta sử dụng toán tử dấu chấm (.).

<Tên-Biến-Cấu-Trúc>.<Tên-Thành-phần>;

Ví dụ : sv.masv; sv.hoten, sv.ngaysinh, sv.diem[1]..., sv.diemtb

Khi đã truy cập tới các thành phần của một biến cấu trúc thì mỗi thành phần đó là 1 biến bình thường và ta có thể gán giá trị hoặc nhập/xuất giá trị cho chúng như cách làm đối với các biến khác.

1.4.2. Mảng - Array

a) Mảng 1 chiều

Cú pháp khai báo : <Kiểu_dữ_liệu> <Tên_mảng>[số phần tử];

Ví dụ : int a[5]; // mảng a kiểu số nguyên, có tối đa 5 phần tử.

Khởi tạo mảng 1 chiều : Chúng ta có thể khởi tạo giá trị cho các phần tử của mảng ngay trong lúc khai báo. Số lượng giá trị khởi gán không được vượt quá kích thước của mảng đã khai báo.

Ví dụ khởi gán sai : `float a [5] = {3.4, 5, 6, 7, 4, 2};`

Khi biên dịch sẽ báo lỗi “ too many initializers for ‘float [5] ‘, nghĩa là “đã khởi gán quá nhiều so với khai báo có 5 phần tử số thực “. Vì vậy cần bỏ bớt 1 dữ liệu, chẳng hạn , sẽ đúng với :

`float a[5]={3.4, 5, 6, 7, 4};`

Khi dùng lệnh cout sẽ hiện :

`a[0] = 3.4 , a[1] =5, a[2]=6, a[3]=7, a[4]=4`

Chú ý : khi nhập dữ liệu từ bàn phím ta có thể cho nhập từ a[1] - > a[n] n là một số nguyên dương, là kích thước của mảng.

Để truy nhập dữ liệu cho các phần tử trong mảng ta cần duyệt tới từng phần tử của mảng .

b) Mảng 2 chiều

Cú pháp khai báo : <Kiểu_dữ_liệu> <Tên_mảng>[số hàng][số cột] ;

Ma trận là dạng đặc trưng của mảng 2 chiều.

Ví dụ : `int a[10][15];` /* mảng a có tối đa 10 hàng và 15 cột */

Truy cập phần tử mảng 2 chiều theo chỉ số hàng, cột, ví dụ phần tử ở hàng i cột j là `a[i][j]`.

c). Mảng cấu trúc

1) Khai báo mảng kiểu cấu trúc

Sau khi có khai báo kiểu và biến cấu trúc, muốn sử dụng mảng cấu trúc thì có thể khai báo theo 1 trong 2 cách giống như khai báo biến cấu trúc :

Cách 1.

```
struct sinhvien
{
    char masv[10];
    string hoten;
    int ngaysinh;
```

```

float diem[10];
float diemtb;
}sv,dslop[60];
// dslop[60] là mảng có 60 phần tử, mỗi phần tử có kiểu cấu trúc sinhvien

```

Cách 2. Khai báo riêng 1 dòng :

```

_struct sinhvien
{
    char masv[10];
    string hoten;
    int ngaysinh;
    float diem[10];
    float diemtb;
};
struct sinhvien sv,dslop[60];

```

Trong dòng khai báo này, **sv** là biến cấu trúc **sinhvien**, **dslop[60]** là tên mảng có kiểu cấu trúc **sinhvien** có tối đa 60 phần tử, mỗi phần tử có kiểu cấu trúc **sinhvien**

2) Truy cập phần tử mảng kiểu cấu trúc

Để truy cập các thành phần của phần tử thứ *i* của một mảng kiểu cấu trúc, ta viết :

Tên_mảng[i].Tên-Thành-Phần;

Ví dụ : dslop[i].masv, dslop[i].hoten, //mã và họ tên sv thứ *i* trong dslop

dslop[10].masv = 230564 dslop[10].hoten = ' Le Thi Duong' ...

dslop[10].ngaysinh = 31072001

Ví dụ : struct sinhvien vl64[60];

Với khai báo này, ta có vl64[46].masv có giá trị là 155764

Vl64[46].hoten sẽ có giá trị là 'Nguyen Tran Thu Phuong'

Bài tập ứng dụng cấu trúc và mảng :

Viết chương trình nhập điểm 3 môn toán, lý, hóa của một lớp có *n* sinh viên, tính tổng điểm 3 môn của từng em.

```

#include <iostream>
#include <conio.h>
using namespace std;
struct sinhvien
{
    char msv[10]; //msv[10] là mảng 10 ký tự
    char hoten[30]; // hoten[30] là mảng 30 ký tự
    double toan, ly, hoa,tong;
}sv,lop[50];
int main()
{
    int n,i;
    cout << "Nhap so sinh vien N: "; cin >> n;
    for (i=1;i<=n;i++)
    {
        cout << "Nhap ho ten sinh vien thu "<<i<<" :";
    }
}

```

```

    cin.ignore();cin.get(lop[i].hoten,30);
    lop[i].tong=0;
    cout<< "Nhap diem 3 mon Toan, Ly, Hoa cua sv thu "<<i<< " : ";
    cin>>lop[i].toan; cin>>lop[i].ly; cin>>lop[i].hoa;
    lop[i].tong = lop[i].toan+lop[i].ly+lop[i].hoa;
}
cout <<" Tong diem cua tung sinh vien :\n";
for (i=1;i<=n;i++)
{
cout<< "Sinh vien"<<lop[i].hoten<<"co tong diem la : "<<lop[i].tong<<"\n";
}
getch();
return 0;
}

```

Bài tập làm tại lớp :

Sử dụng kiểu string masv, hoten viết lại chương trình trên cho kiểu cấu trúc :
 struct sinhvien

```

    { string masv, hoten; //msv và hoten có kiểu string
      double toan, ly, hoa,tong;
    }sv,lop[50];

```

Hướng dẫn : thêm **#include <string>** vào phần đầu chương trình, sử dụng **cin.ignore(); getline(cin, Tên_mảng[i].Tên-Thành-Phần);**//khi nhập dữ liệu **fflush(stdin);**);//xóa bộ nhớ đệm

Chương trình tham khảo : dùng biến kiểu string cho chuỗi ký tự ;

```

#include <iostream>
#include <conio.h>
#include <string>
using namespace std;

```

```

struct sinhvien

```

```

    { string msv, hoten;
      double toan, ly, hoa,tong;
    }sv,lop[50];

```

int main() //Những chữ tô **đỏ** là dùng cho kiểu **string**, khác với chương trình trên

```

{ int n,i;
  cout << "Nhap so sinh vien N: "; cin >>n;
  for (i=1;i<=n;i++)
  { cout <<"Nhap ho ten sinh vien thu "<<i<<" :";
    cin.ignore();
    getline(cin,lop[i].hoten);
    fflush(stdin);
    lop[i].tong=0;
    cout<< "Nhap diem 3 mon Toan, Ly, Hoa cua sv thu "<<i<< " : ";
    cin>>lop[i].toan;    cin>>lop[i].ly;    cin>>lop[i].hoa;

```

```

        lop[i].tong = lop[i].toan+lop[i].ly+lop[i].hoa;
    }
    cout <<" Tong diem cua tung sinh vien :\\n";
for (i=1;i<=n;i++)
{ cout<< " Sinh vien  " <<lop[i].hoten<< " co tong diem la : "<<lop[i].tong<<"\\n";
  }
  getch();
  return 0;
}

```

BÀI TẬP VỀ NHÀ

Bài 1. Vận dụng chương trình ví dụ đã học và làm trên lớp về mảng cấu trúc, viết chương trình quản lý thư viện, mỗi quyển sách cần lưu các thông tin tên sách, tên tác giả, nhà xuất bản, năm xuất bản. Sau khi nhập sách, yêu cầu tìm, hiện những sách đã xuất bản vào **năm** nhập từ bàn phím.

Hướng dẫn.

Các em tùy chọn 1 trong 2 kiểu string hay kiểu mảng ký tự char khi khai báo biến tên sách, tác giả, nhà xuất bản. Cấu trúc books có dạng như sau :

```

struct books
{ char tensach[30]; //hoặc string tensach ;
  char tacgia[30]; // hoặc string tacgia ;
  char nhaxb[20]; // hoặc string nhaxb – Nhà xuất bản ;
  int namxb;//năm xuất bản
}thuvien[100];

```

Chương trình có các phần : nhập sách , nhập năm cần tìm sách(có thể dùng biến year để lưu năm cần tìm được nhập vào từ bàn phím), tìm và hiện lên những sách đã được xuất bản vào một năm đã nhập.

Bài 2. Dùng kiểu Cấu trúc viết chương trình :

- Nhập danh sách N sinh viên , mỗi sinh viên có các thành phần dữ liệu : mã sinh viên, họ tên, điểm m môn, điểm trung bình .
- Tính điểm trung bình của từng sinh viên.
- Hiện lại danh sách theo từng dòng, mỗi dòng có mã sv, họ tên, điểm trung bình.

Gợi ý: tham khảo kiểu cấu trúc sinhvien trong bài giảng để làm chương trình, thay 3 môn toán, lý, hóa bằng diemmon[15] (điểm của các môn trong học kỳ).

1.4.3.Con trỏ và bộ nhớ động

1.4.3.1. Một số khái niệm dẫn nhập

Trước khi nói đến con trỏ, cần giải thích một số khái niệm liên quan đến việc lưu trữ dữ liệu của biến trong bộ nhớ máy tính.

a)Bộ nhớ ảo vào bộ nhớ vật lý (Virtual memory & Physical memory)

Việc cấp phát bộ nhớ, truy xuất dữ liệu cho các biến trong chương trình được thực hiện bởi thiết bị phần cứng có tên là Memory Management Unit(MMU) và một chương trình định vị địa chỉ bộ nhớ gọi là Virtual address space, chúng hoạt động theo nhiệm vụ của Hệ điều hành.

Địa chỉ của biến mà ta nhìn thấy thực ra là địa chỉ ở vùng nhớ ảo (virtual memory)- thường khởi đầu từ địa chỉ 1000 trong vùng lưu dữ liệu trên máy tính.

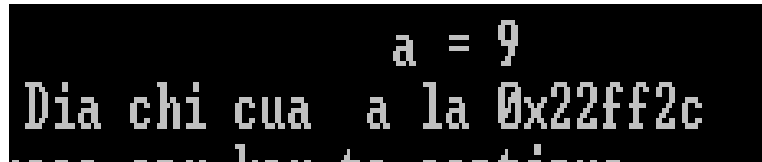
b) Toán tử lấy địa chỉ của biến &

Trong C++, hàm &a sẽ cho địa chỉ ảo của biến a.

Ví dụ, với chương trình sau :

```
#include <iostream>
using namespace std;
int main()
{ cout << "\n  Hàm & : " << endl;
  int a=9;
  cout<<"          a = "<<a<<"\n";
  cout<<" Địa chỉ của a là "<<&a<<"\n";
  system("pause");
  return 0;
}
```

Khi chạy sẽ cho kết quả là :



c) Tham chiếu - Reference

Việc tham chiếu trong C++ là tạo ra một biến khác cùng kiểu dữ liệu, cùng tham chiếu đến vùng nhớ như biến được tham chiếu.

Để tạo ra một biến tham chiếu **x_ref** đối với biến **x** ta đặt toán tử **&** ngay trước **x_ref** khi khai báo biến **&x_ref**. Địa chỉ của biến tham chiếu sẽ được xác định, không thay đổi, những xử lý làm thay đổi giá trị **x_ref** đều thay đổi giá trị **x**.

Ví dụ :

```
#include <iostream>
using namespace std;
int main()
{      cout << "\n    Ham Tham chieu & : " << endl;
      int x=9;
      int &x_ref = x;
      cout<<"    Dia chi cua  x = "<<&x<<"\n";
      cout<<"    Dia chi cua bien tham chieu: "<<&x_ref<<"\n";
      system("pause");
      return 0;
}
```

Kết quả chạy chương trình :

```
Ham Tham chieu & :  
Dia chi cua x = 0x22ff28  
Dia chi cua bien tham chieu: 0x22ff28
```

d) Toán tử * dấu hoa thị

Ngoài việc lấy giá trị của một biến theo tên biến, ta còn có thể lấy giá trị của một ô nhớ theo địa chỉ cụ thể của nó bằng toán tử dấu hoa thị “*”.

Toán tử “*” không chỉ cho giá trị của một địa chỉ ô nhớ mà còn cho phép thay đổi giá trị của ô nhớ đó bằng lệnh gán lại giá trị cho ô nhớ mà không cần thông qua tên biến :

$*(\&\text{biến}) = \text{giá trị mới};$

Ví dụ :

```
#include <iostream>  
using namespace std;  
int main()  
{ cout << "\n Toan tu * : " << endl;  
  int a=9;  
  cout<<" Gia tri cua a = "<<a<<"\n";  
  cout<<" Dia chi cua a = "<<&a<<"\n";  
  cout<<" Gia tri o dia chi &a : "<<*(&a)<<"\n";  
  *(&a) = 19;  
  cout<<" 2)Gan gia tri moi vao o nho *(&a) = 19; \n";  
  cout<<" 2)Gia tri moi cua a = "<<a<<"\n";  
  system("pause");  
  return 0;  
}
```

Kết quả chạy chương trình :

```
Toan tu * :  
1)Gia tri cua a = 9  
Dia chi cua a = 0x22ff2c  
Gia tri o dia chi &a : 9  
  
2)Gan gia tri moi vao o nho *(&a) = 19;  
Gia tri moi cua o nho = 19
```

1.4.3.2.Con trỏ

a)Bộ nhớ tĩnh

a1) Biến toàn cục(global variable), biến cục bộ (local variable), biến static

Khi học về Hàm, trong C++ có 3 loại biến :

- Biến toàn cục (**global variable**) được khai báo bên ngoài khối lệnh, bên ngoài hàm, có thể được truy xuất tại bất cứ dòng lệnh nào đặt bên dưới biến đó. Biến toàn cục tồn tại đến khi chương trình kết thúc.

- Biến cục bộ (**local variable**) được khai báo bên trong khối lệnh, bên trong hàm, có thể được truy xuất tại bất cứ dòng lệnh nào đặt bên dưới biến đó và trong cùng khối lệnh, bên trong hàm. Biến cục bộ bị hủy khi chương trình chạy ra ngoài khối lệnh, ra khỏi hàm chứa biến đó.
- Biến tĩnh được khai báo với từ khóa **static**, vùng hoạt động giống như biến toàn cục nhưng khai báo ở đâu cũng được.

a2) Cấp phát bộ nhớ tĩnh

Tương ứng với các kiểu biến này là 2 cách thức cấp phát bộ nhớ cho chương trình trên bộ nhớ ảo:

- ✓ **cấp phát bộ nhớ tĩnh**, được áp dụng cho biến **static** và biến toàn cục, vùng nhớ và kích thước của vùng nhớ được cấp phát tại thời điểm biên dịch chương trình
- ✓ **cấp phát bộ nhớ tự động** được sử dụng để cấp phát vùng nhớ cho các biến cục bộ, các tham số của hàm, bộ nhớ được cấp phát tại thời điểm chương trình đang chạy, khi chương trình đi vào một khối lệnh, các vùng nhớ được cấp phát sẽ được thu hồi khi chương trình đi ra khỏi một khối lệnh.

Cả 2 phương thức cấp bộ nhớ trên đây đều là tĩnh, có nhược điểm là :

Kích thước vùng nhớ được cung cấp tại thời điểm biên dịch chương trình và tồn tại trong suốt thời gian chương trình chạy, khi dùng một mảng lớn mà dữ liệu ít sẽ gây dư thừa bộ nhớ đã cấp phát;

Phân vùng bộ nhớ **stack** được dùng cho các phương thức cấp phát bộ nhớ tĩnh và tự động nói trên bị chặn trên về kích thước tùy theo hệ điều hành, thường là 1MB (tương đương khoảng 1024 Kilobytes hay 1024*1024 bytes). Hạn chế này thường gây ra lỗi **stack overflow** khi chương trình yêu cầu cấp phát vùng nhớ vượt quá dung lượng của **stack**

Phân vùng trên bộ nhớ ảo :

Địa chỉ cao 0x0000FFFF	Stack	dùng để cấp phát bộ nhớ cho tham số của các hàm và biến cục bộ
	Heap	được sử dụng để cấp phát bộ nhớ thông qua kỹ thuật Dynamic memory allocation.
	BSS (Banque de Donnees du Sous-Sol) - Uninitialized Data Segment	dùng để lưu trữ các biến kiểu static, biến toàn cục (global variable) nhưng chưa được khởi tạo giá trị cụ thể
	Data	sử dụng để khởi tạo giá trị cho các biến kiểu static, biến toàn cục (global variable)
	Text (Code segment)	lưu trữ các mã lệnh đã được biên dịch của các chương trình

b) Bộ nhớ động

Để khắc phục các nhược điểm của cấp phát bộ nhớ tĩnh đã phân tích ở trên, cách thức cấp phát bộ nhớ động đã được áp dụng, là giải pháp cấp phát bộ nhớ cho chương trình tại thời điểm chương trình đang chạy (run-time), sử dụng phân vùng **Heap** trên bộ nhớ ảo để cấp phát cho chương trình. **Heap** là phân vùng có dung lượng bộ nhớ ảo lớn nhất, có thể lên đến 1GB và không phụ

thuộc vào hệ điều hành mà chỉ phụ thuộc vào cấu trúc phần cứng của máy (như dung lượng các thanh RAM).

Với kỹ thuật cấp phát bộ nhớ động tại thời điểm run-time, không có biến mới nào được tạo ra mà chỉ có thể sử dụng vùng nhớ mới. Đó là lý do phát sinh kỹ thuật dùng con trỏ để quản lý vùng nhớ trên Heap, con trỏ sẽ lưu giữ địa chỉ đầu tiên của vùng nhớ được cấp phát.

c) Con trỏ

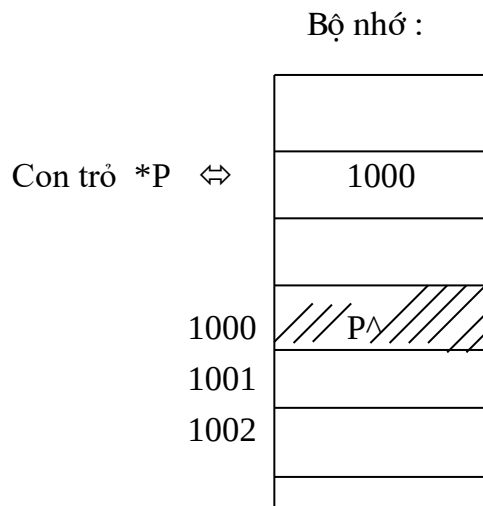
Những dữ liệu được xác định trước bởi khai báo tên biến trong chương trình và tồn tại trong suốt quá trình chương trình thực hiện gọi là **dữ liệu tĩnh**. Dữ liệu mà có thể được sinh ra và mất đi trong quá trình thực hiện chương trình gọi là **dữ liệu động**. Việc truy nhập dữ liệu động phải thông qua một biến con trỏ, trỏ tới bộ nhớ của biến động.

Định nghĩa : một con trỏ (pointer) là một biến được dùng để lưu trữ địa chỉ của biến khác, con trỏ có địa chỉ độc lập so với địa chỉ vùng nhớ mà nó trỏ đến.

Như vậy, giá trị của con trỏ chính là địa chỉ của biến (hoặc địa chỉ ảo) mà nó trỏ tới.

Ký hiệu *p là biến kiểu con trỏ, p[^] sẽ là biến động, chứa giá trị dữ liệu ở vùng nhớ mà con trỏ p chỉ đến.

Chúng ta có thể mô tả con trỏ như sau :



Trong sơ đồ trên, con trỏ *P có giá trị là 1000 - là địa chỉ của ô nhớ mà chứa nội dung của biến động P[^].

Trong thực tế chúng ta ít quan tâm đến địa chỉ thực của con trỏ *P (trong ví dụ trên, là địa chỉ ô nhớ có giá trị 1000) mà chỉ là nội dung của nó, tức là địa chỉ của biến động P[^] (là giá trị 1000).

Khi con trỏ chưa được định nghĩa hay rõ ràng thì biến động P[^] là chưa được xác định.

Khai báo con trỏ

Kiểu con trỏ là một kiểu trỏ đến (chỉ đến) một vùng chứa dữ liệu của các biến động, các biến này có thể có các kiểu dữ liệu như int, float, char, v.v...

Giống như các kiểu dữ liệu đã xét, trong chương trình nếu có sử dụng kiểu con trỏ, nó phải được khai báo như các kiểu dữ liệu khác bằng lệnh sau:

<kiểu dữ liệu> *<tên biến con trỏ> ;

Trong đó:

+ **kiểu dữ liệu**: là một trong các kiểu dữ liệu như int, float, char,...

- + *dấu ** : đứng tr-ớc tên biến để chỉ biến đó là biến kiểu con trỏ.
- + *tên biến con trỏ*: là tên đ-ợc đặt theo qui tắc đặt tên của ngôn ngữ .

Ví dụ: `int *ip;` // con trỏ kiểu integer
`float *fp;` // con trỏ kiểu float
`char *chp;` // con trỏ kiểu char
`string *strp;` // con trỏ kiểu string

Kích thước trong bộ nhớ của con trỏ các loại kiểu dữ liệu.

Khi dùng con trỏ các kiểu dữ liệu, trong máy 32 bit, kích thước của chúng là 4 bytes. Xem đoạn chương trình sau :

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{ cout << "Hello world!\n" << endl;
  cout << setw(20) << "Kích thước của CON TRO char trong bộ nhớ là : "<< sizeof(char*) << endl;
  cout << setw(20) << "Kích thước của CON TRO int trong bộ nhớ là : "<< sizeof(int*) << endl;
  cout << setw(20) << "Kích thước của CON TRO double trong bộ nhớ là : "<< sizeof(double*) << endl;
  cout << setw(20) << "Kích thước của CON TRO string trong bộ nhớ là : "<< sizeof(string*) << endl;
  return 0;
}
```

Kết quả chạy chương trình :

```
Kích thước của CON TRO char trong bộ nhớ là :4
Kích thước của CON TRO int trong bộ nhớ là :4
Kích thước của CON TRO double trong bộ nhớ là :4
Kích thước của CON TRO string trong bộ nhớ là :4
```

1.4.3.3. Các thao tác đối với con trỏ

a) *Cấp phát và thu hồi bộ nhớ đối với con trỏ*

Việc cấp phát bộ nhớ cần thực hiện qua 2 bước:

- Yêu cầu cấp phát vùng nhớ trên **Heap** bằng toán tử **new**
- Lưu trữ địa chỉ của vùng nhớ vừa được cấp phát bằng con trỏ.

Việc thu hồi vùng nhớ đã cấp phát cho con trỏ trên Heap khi kết thúc khối lệnh được thực hiện bằng toán tử **delete**.

Toán tử new trong chuẩn C++11 được định nghĩa với **3 prototype** như sau:

```
void* operator new (std::size_t size);
void* operator new (std::size_t size, const std::nothrow_t& nothrow_value) noexcept;
void* operator new (std::size_t size, void* ptr) noexcept;
```

Ở đây cần chú ý kiểu trả về của hàm void *.

Toán tử new sau khi xin cấp phát vùng nhớ trên **Heap** sẽ trả về một con trỏ kiểu **void** chứa **địa chỉ của vùng nhớ được cấp phát** (nếu cấp phát thành công). Có thể gán giá trị trả về của toán tử **new** cho một con trỏ khác để quản lý vùng nhớ đã được cấp phát.

Cú pháp sử dụng toán tử new : new <kiểu dữ liệu>;

Ví dụ : new int;// cấp 4 bytes trên Heap cho một biến kiểu int

 new double;// cấp 8 bytes trên Heap cho một biến kiểu double

Sau khi được cấp phát vùng nhớ, sẽ có địa chỉ vùng nhớ được trả về, cần gán nó cho con trỏ cùng kiểu để quản lý :

```
int *p1= new int;
```

```
double *p2= new double;
```

Trong 2 dòng ví dụ trên, 2 vùng nhớ được cấp phát sẽ được quản lý bởi 2 con trỏ *p1 và *p2, thông qua *p1 và *p2 có thể thay đổi giá trị bên trong 2 vùng nhớ này.

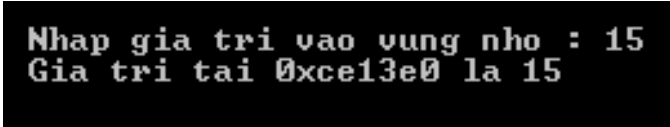
Ví dụ: int *p1= new int;

```
cout<<"    Nhập giá trị vào vùng nhớ :";
```

```
cin>>*p1;
```

```
cout<<"    Giá trị tại "<<p1<<" là "<<*p1;
```

Kết quả chạy đoạn chương trình :



```
Nhap gia tri vao vung nho : 15
Gia tri tai 0xce13e0 la 15
```

Có thể vừa cấp phát bộ nhớ vừa khởi tạo giá trị tại vùng nhớ đó cho một biến đơn:

```
int *p1= new int(10);
```

```
int *p2= new int{*p1};
```

Ví dụ :

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{    cout << "    Cap phat bo nho pointer \n" << endl;
```

```
     int *p1= new int;
```

```
     cout << "    Nhap gia tri vao vung nho : ";
```

```
     cin>>*p1;
```

```
     cout << "    Gia tri tai p1 " <<p1<< " la " <<*p1<<"\n";
```

```
     delete p1;
```

```
     int *p2=new int(10);
```

```
     int *p3=new int{*p2};
```

```
     cout << "    Gia tri tai p2 " <<p2<< " la " <<*p2<<"\n";
```

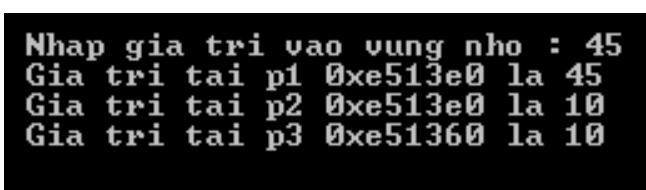
```
     cout << "    Gia tri tai p3 " <<p3<< " la " <<*p3<<"\n";
```

```
     system("pause");
```

```
     return 0;
```

```
}
```

Kết quả chạy chương trình :



```
Nhap gia tri vao vung nho : 45
Gia tri tai p1 0xe513e0 la 45
Gia tri tai p2 0xe513e0 la 10
Gia tri tai p3 0xe51360 la 10
```

Cú pháp toán tử delete

Khi không sử dụng tiếp vùng nhớ đã được cấp phát cho con trỏ trên **Heap**, cần trả lại vùng nhớ đó cho hệ điều hành. Để xóa một vùng nhớ đã cấp cho con trỏ p, ta viết :

```
delete p;
```

Sau lệnh này, nếu hệ điều hành chưa cấp phát vùng nhớ gắn với con trỏ p cho biến khác thì vẫn có thể dùng con trỏ p để thay đổi giá trị bên trong nó.

Sử dụng toán tử delete không có nghĩa là **delete** tất cả mọi thứ bên trong vùng nhớ mà con trỏ trỏ đến. Toán tử **new** và **delete** chỉ mang ý nghĩa về "quyền sử dụng" vùng nhớ trên Stack hoặc trên Heap do hệ điều hành có quyền trao. Khi delete vùng nhớ được cấp phát được trả lại cho hệ điều hành quản lý, nhưng con trỏ vẫn còn trỏ vào địa chỉ đó, toán tử **delete** không tác động gì đến con trỏ, giá trị trên vùng nhớ gắn với nó có thể vẫn còn giữ nguyên khi chưa có chương trình nào can thiệp vào.

Con trỏ bị treo

"Con trỏ bị treo" thường xảy ra sau khi giải phóng vùng nhớ bằng toán tử **delete**. Do sau toán tử **delete**, con trỏ vẫn còn trỏ vào địa chỉ đó, nếu sử dụng toán tử lấy giá trị của một ô nhớ theo địa chỉ cụ thể của nó bằng toán tử dấu hoa thị ***(dereference)** cho con trỏ tại thời điểm này sẽ gây ra lỗi **undefined behavior**.

b) Gán giá trị cho con trỏ (=)

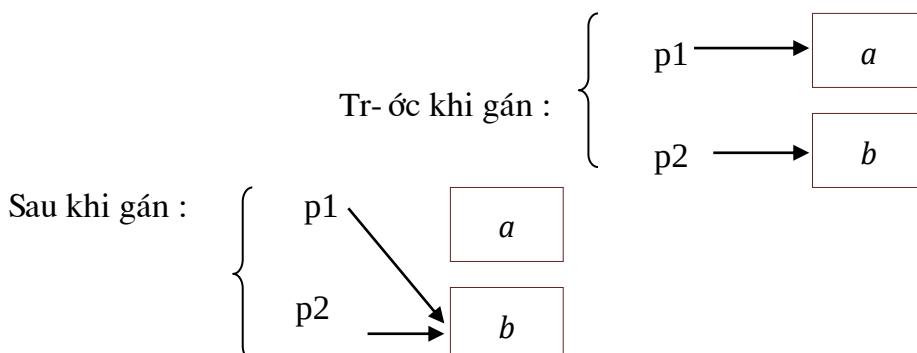
Khi muốn gán giá trị cho một con trỏ để nó trỏ đến địa chỉ ô nhớ của biến nào đó, ta có thể thực hiện các câu lệnh sau :

```
int *p; //khai báo con trỏ p kiểu int
int a=10; //khai báo biến a kiểu int và gán giá trị = 10
p=&a; //con trỏ p sẽ trỏ đến địa chỉ ô nhớ chứa biến a
```

c) Gán 2 con trỏ cùng kiểu.

```
+ Ví dụ:   int    a=10,b=20 ;
           int    *p1,*p2 ;
           p1=&a; // p1 trỏ đến a
           p2=&b; // p2 trỏ đến b
           p1=p2;
```

Lệnh gán này sẽ làm cho hai con trỏ p1 và p2 cùng chỉ đến một vùng chứa dữ liệu b=20.



c) So sánh hai biến con trỏ cùng kiểu: Ta chỉ có thể so sánh hai biến con trỏ cùng kiểu bằng các phép so sánh == (bằng) và != (khác).

+ Ví dụ:

```
int *p1,*p2;
if (p1== p2)cout<<"p1 bằng p2";
```

d)Hàng con trỏ NULL: Null là một giá trị hằng đặc biệt dành cho biến con trỏ để báo con trỏ không chỉ vào đâu cả. Nếu ta gán hằng Null này cho một biến con trỏ thì có nghĩa biến đó không chỉ vào đâu cả.

+ Ví dụ:

```
int *p;
p = null; { a không chỉ vào địa chỉ nào }
```

e)Chú ý:

Khi dùng con trỏ trỏ đến các phần tử của dãy, giá trị của nó thay đổi theo :

Ví dụ :

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{   cout << "\n\n Hien cac gia tri con tro :\n\n" << endl;
    int *p;
    int x[6] = { 10, 15, 20, 25,30,35};
    cout<<"    Gia tri con tro tai cac x[i] la :\n";
    for(int i = 0; i < 6; i++)
        { p = &x[i];
          cout <<setw(20)<<" p tai x["<<i<<"] = "<< p << endl;
        }
    cout<<"\n\n";
    system("pause");
    return 0;
}
```

1.4.3.4. Con trỏ mảng

a) Cấp phát vùng nhớ động cho con trỏ mảng

Để cấp phát vùng nhớ động cho con trỏ mảng một chiều trên **Heap**, ta cũng sử dụng toán tử **new**

Cú pháp : **new** <kiểu dữ liệu>[số phần tử];

Khi cấp phát thành công, toán tử **new** sẽ trả về địa chỉ của phần tử đầu tiên của vùng nhớ được cấp phát, và 1 con trỏ có kiểu dữ liệu đã khai báo lưu trữ địa chỉ trả về để quản lý vùng nhớ mảng.

Chú ý : địa chỉ mà con trỏ mảng vừa được khởi tạo lưu trữ chính là địa chỉ của phần tử đầu tiên trong mảng. Cần phân biệt sự khác nhau giữa con trỏ mảng và mảng các con trỏ. Trong mục này chỉ xét **con trỏ mảng**.

Ví dụ:

```
int *p = new int[5];
for(int i=0;i<5;i++)
{cout<<"    Nhập giá trị cho phần tử thu "<<i<<" của mảng :";
  cin>> *(p + i) ;
}
cout<<"    Các giá trị của mảng đã nhập :";
for(int i=0;i<5;i++) cout<< *(p + i)<<" ";
```

Kích thước của mảng con trỏ không nhất thiết phải xác định khi lập trình mà có thể nhập vào khi chương trình đang chạy, ví dụ

```
int n;
cout<<"    Nhập số lượng phần tử của mảng :";
cin>>n;
int *p= new int[n];
```

b) Giải phóng mảng vùng nhớ động

Cú pháp : **delete[]** <tên con trỏ>;

Để giải phóng mảng vùng nhớ liên tục được cấp phát trên **Heap**, cần thêm cặp ngoặc vuông []

Ví dụ :

```
int *p= new int[5];
....
...
delete[] p;
```

Kết hợp các phần trên, ta có một chương trình ví dụ về cấp phát, nhập giá trị, giải phóng bộ nhớ động đối với con trỏ mảng :

```

#include <iostream>
#include <iomanip>
using namespace std;
int main()
{   cout << "\n  CON TRO MANG - ARRAY POINTER \n" << endl;
    int i,n;
    cout<<"      Nhap so luong phan tu Array : ";
    cin>>n;
    int *p = new int[n];

    for(i=0;i<n;i++)
        {cout<<"      Nhap gia tri phan tu thu "<<i<<" cua array :";
          cin >> *(p + i );
        }
    cout<<"      Cac gia tri phan tu Array da nhap : \n";
    cout<<setw(20);
    for(i=0;i<n;i++) {cout << *(p + i)<<" ";}
    delete[] p;
    system("pause");
    return 0;
}

```

Kết quả chạy chương trình :

```

CON TRO MANG - ARRAY POINTER

Nhap so luong phan tu Array : 5
Nhap gia tri phan tu thu 0 cua array :1
Nhap gia tri phan tu thu 1 cua array :2
Nhap gia tri phan tu thu 2 cua array :3
Nhap gia tri phan tu thu 3 cua array :4
Nhap gia tri phan tu thu 4 cua array :5
Cac gia tri phan tu Array da nhap :
      1 2 3 4 5 _

```

Bài tập :

- 1) Làm các bài tập đã nêu trong bài
- 2) Nhập và chạy lại các chương trình có trong bài

1.5.Cấu trúc dữ liệu class và đối tượng

1.5.1.Khái niệm về Lớp – class, đối tượng, thuộc tính, phương thức

Ngôn ngữ C++ là ngôn ngữ lập trình hướng đối tượng. Một đối tượng(Object) được xác định bởi các dữ liệu(du_lieu) và các phương thức(method) thao tác các dữ liệu đó, các phương thức là các hàm.

Lớp – class là bản thiết kế của các đối tượng, là một kiểu dữ liệu của đối tượng do người dùng định nghĩa, chứa các thành viên dữ liệu và các hàm thành viên của đối tượng đó.

Như vậy, trong một lớp có :

- Thuộc tính (Attribute) là các thành viên dữ liệu – là các biến
- Phương thức (Method) là các hàm thành viên dùng để thao tác với các thành viên dữ liệu

Ví dụ.

Một lớp sinh viên có các thành phần :

Thuộc tính(Attribute) : mã sinh viên, họ tên, lớp, điểm các môn trong học kỳ,...

Phương thức (Method): dựa vào các dữ liệu trong phần thuộc tính sẽ lập các hàm phương thức tính điểm trung bình , hàm phân loại học lực cho từng sinh viên, hàm thống kê số sinh viên theo các loại học lực ...

Định nghĩa lớp – class trong C++

Cú pháp :

```
class <tên lớp>
{
    Khai báo các thành viên dữ liệu (có thể dùng private và
    public)
    như tên biến, mảng, con trỏ có kiểu chuẩn
    ( int, float, char, char*, long, struct,class...)

    Khai báo các hàm
};
```

Chú ý :

Trong phần Khai báo các thành viên dữ liệu như tên biến, mảng, con trỏ có kiểu chuẩn (int, float, char, char*, long, struct,class...) có thể dùng private(riêng) và public(công cộng), nếu không khai báo thì được hiểu là private. Những thành viên private chỉ được dùng trong thân các hàm phương thức của lớp, các thành viên public được sử dụng cả bên trong và bên ngoài lớp.

Thường thì các thành viên dữ liệu được khai báo private để an toàn dữ liệu, còn các hàm lại khai báo public để các hàm khác trong chương trình có thể gọi tới.

Một phương thức cũng có thể được khai báo private, khi đó nó chỉ được sử dụng trong thân của các phương thức cùng lớp.

Các hàm phương thức có thể viết codes tạo hàm ngay trong định nghĩa lớp nếu ít dòng, nếu nhiều dòng thì có thể viết codes ở ngoài định nghĩa lớp. Khi tạo hàm bên trong định nghĩa lớp thì quy

cách viết hàm giống như một hàm thông thường, khi tạo bên ngoài định nghĩa lớp thì phải viết thêm

<tên lớp>::

ở trước tên hàm phương thức, để chương trình nhận biết hàm này dùng trong phạm vi lớp nào.

Trong thân hàm phương thức của một lớp có thể sử dụng các thuộc tính và các hàm phương thức của lớp đó hay sử dụng hàm tự lập trong chương trình; kiểu giá trị trả về của phương thức là một trong các kiểu chuẩn int, float, char, char*, long, hay struct, class.

1.5.2. Biến và mảng đối tượng

1. Khai báo biến và mảng đối tượng

Sau khi được định nghĩa, một lớp được coi như một kiểu dữ liệu đối tượng và được dùng để khai báo các biến, mảng đối tượng.

Cú pháp :

```
<tên lớp> <danh sách đối tượng>;  
<tên lớp> <danh sách mảng đối tượng>;
```

2. Truy cập các thành viên dữ liệu của đối tượng

Thành viên dữ liệu là phần thuộc tính của đối tượng, muốn truy cập vào thuộc tính của đối tượng nào phải dùng toán tử dấu chấm . kèm sau tên đối tượng :

```
<tên đối tượng>.<thuộc tính>  
<tên mảng đối tượng>[chỉ số].<thuộc tính>
```

3. Lời gọi hàm phương thức của đối tượng

Khi có lời gọi hàm phương thức của đối tượng nào, cần viết tên đối tượng kèm dấu chấm trước tên hàm phương thức.

```
<tên đối tượng>.<tên hàm phương thức>  
<tên mảng đối tượng>[chỉ số].<tên hàm phương thức>
```

Ví dụ . Có lớp point :

```
class point  
{  
private:  
    int x, y, color;  
public:  
    void nhapdl();  
    void hien();  
    void an()  
{  
    putpixel(x,y,getbkcolor());
```

```

    }
};

void point ::nhapdl()
{
    cout<<" Nhập tung độ y và hoành độ x của điểm : ";cin>>y>>x;
    cout <<" Nhập mã màu : ";cin>>color;
}

```

Khai báo point d1, d2, d[10];

Khi đó có thể viết :

d1.nhapdl();//Lời gọi hàm nhập dữ liệu cho đối tượng d1

d[1].nhapdl();//Lời gọi hàm nhập dữ liệu cho đối tượng thứ nhất của mảng đối tượng d

1.3.3.Con trỏ đối tượng

1.Khai báo và sử dụng con trỏ đối tượng

Con trỏ đối tượng dùng để chứa địa chỉ của biến, mảng đối tượng.

Cú pháp khai báo con trỏ đối tượng :

```
<tên lớp> *<tên con trỏ> ;
```

Ví dụ :

```
sinhvien *p1, *p2, *p3;//Khai báo 3 con trỏ p1, p2, p3
```

```

<Tên đối tượng>.<Tên thuộc tính>
<Tên con trỏ> -> <Tên thuộc tính>
<Tên mảng đối tượng>[i]. <Tên thuộc tính>
<Tên con trỏ>[i]. <Tên thuộc tính>

```

```
sinhvien sv1, sv2;//Khai báo 2 đối tượng sv1, sv2
```

```
sinhvien sv[50];//Khai báo mảng đối tượng sv có tối đa 50 đối tượng
```

Để nói rằng con trỏ p1 trỏ đến đối tượng sv1, viết :

```
p1=&sv1;// con trỏ p1 chứa địa chỉ của sv1
```

Để nói rằng con trỏ p2 trỏ đến đầu mảng đối tượng sv, viết :

```
p2=sv;
```

Để nói rằng tạo một đối tượng mới và lưu địa chỉ của nó vào con trỏ p3, viết :

```
p3= new sinhvien;
```

Khi dùng con trỏ, để biểu thị thuộc tính của đối tượng sẽ dùng ký hiệu -> hay dấu chấm . đi kèm với tên đối tượng hoặc tên con trỏ :

2.Con trỏ *this*

This là con trỏ ngầm định chứa địa chỉ của đối tượng hiện hành, được sử dụng trong hàm phương thức đang tác động vào thuộc tính của đối tượng này.

Trong các hàm được viết trong định nghĩa của lớp, trước tên thuộc tính không cần viết tên đối tượng là nhờ con trỏ *this* đóng vai trò đối tượng hiện hành.

Ví dụ :

Cho lớp .

```
class sinhvien
{
public:
string masv, hoten, lop;
float diem[10], tb;
char ploai;
public:
void nhapdl();//nhập dữ liệu và tính điểm trung bình
void ploaiHL();//Phân loại học lực A, B, C, D, F
{
    if(this->tb>=8.5) {this->ploai='A';}
    else
        { if((this->tb<8.5)&&( this->tb>=7.0)) { this->ploai='B';}
          else
              { if((this->tb<7)&&( this->tb>=5.5)) { this->ploai='C';}
                else
                    { if((this->tb<5.5)&&( this->tb>=4)) { this->ploai='D';}
                      else
                          this->ploai='F';
                    }
                }
            }
        }
    }
};//Hết định nghĩa lớp sinhvien
```

Trong chương trình chính main(), có mảng đối tượng sv[50] có kiểu lớp sinhvien, khi có lời gọi hàm phương thức sv[i].ploaiHL(), địa chỉ của sv[i] sẽ truyền cho con trỏ **this** :

this = &sv[i]

nên this->tb chính là sv[i].tb, this->ploai chính là sv[i].ploai

3. Các đối số khác của phương thức

Khi học về Hàm đều biết mỗi hàm thường phải có đối số, với hàm phương thức của đối tượng cũng vậy, nó có thể có nhiều đối số, this chính là đối số đầu tiên của hàm phương thức, ở dạng ngầm định.

Ngoài ra, các hàm phương thức có thể có các đối số khác với các kiểu dữ liệu chuẩn hay ngoài chuẩn như tên các đối tượng.

Kiểu dữ liệu chuẩn là int, float, char, string,..., con trỏ tham chiếu đến int*, float*, char*...

Kiểu dữ liệu ngoài chuẩn : cấu trúc, lớp, đối tượng, hợp, enum,..., con trỏ tham chiếu đến các kiểu ngoài chuẩn này.

Một số chú ý:

- Thuộc tính của một lớp có thể là đối tượng của lớp đã được định nghĩa trước đó;
- Phương thức có thể có giá trị trả về là kiểu đối tượng hay con trỏ đối tượng;
-

Ví dụ . Quản lý sinh viên

Trở lại ví dụ : Một lớp sinhvien có các thành phần :

Thuộc tính(Attribute) : mã sinh viên, họ tên, lớp, điểm các môn trong học kỳ, điểm trung bình...

Phương thức (Method): dựa vào các dữ liệu trong phần thuộc tính sẽ lập các hàm phương thức nhập dữ liệu; tính điểm trung bình; phân loại học lực cho từng sinh viên.

Ta tạo lớp sinh viên với chương trình như sau.

```
#include <iostream>
#include <string>
#include <iomanip>
using namespace std;
class sinhvien
{ public:
string masv, hoten,lop;
float diem[10], tb;
char ploai;
public:
void nhapdl();//nhập dữ liệu và tính điểm trung bình
void ploaiHL();//Phân loại học lực A, B, C, D, F
    {if(this->tb>=8.5) {this->ploai='A';}
      else
        { if((this->tb<8.5)&&( this->tb>=7.0)) { this->ploai='B';}
          else
            { if((this->tb<7)&&( this->tb>=5.5)) { this->ploai='C';}
              else
                { if((this->tb<5.5)&&( this->tb>=4)) { this->ploai='D';}
                  else    this->ploai='F';
                }
            }
          }
        }
    }
};//Hết định nghĩa lớp sinhvien

void sinhvien::nhapdl();// nhập dữ liệu sv và tính điểm tb
{ int j,m;
cout<<"  Nhập so mon hoc (m): ";cin>>m;
cout<<" \n  Nhập ma sinh vien : ";
cin.ignore();getline(cin,this->masv);//xóa bộ nhớ đệm; loại bỏ dấu trống ở đầu
fflush(stdin); cout<<"  Nhập ho ten sinh vien : ";
cin.ignore(0);getline(cin,this->hoten);//xóa bộ nhớ đệm; loại bỏ dấu trống ở đầu
fflush(stdin); cout<<"  Nhập ten lop : ";
cin.ignore(0);getline(cin,this->lop);//xóa bộ nhớ đệm; loại bỏ dấu trống ở đầu
fflush(stdin); float tb0=0;
for(j=1;j<=m;j++)
{cout<<"  Nhập diem mon thu "<<j<<" = ";cin>>this->diem[j];
tb0=tb0+this->diem[j];
}
this->tb=tb0/m;
}
```

```

int main()
{
    int i,n;
    cout<<" \n Nhap so luong sinh vien n = ";cin>>n;
    sinhvien sv[50];
    for(i=1;i<=n;i++)
    {cout<<" \n-----";
        cout<<" \n Nhap so lieu sinh vien thu "<<i<<" : \n";
        sv[i].nhapdl();
        sv[i].ploaiHL();
    }

    cout<<" \n-----";
    cout<<" \n      HIEN DANH SACH KET QUA      \n";
    cout<<" \n   Ma sv |   Ho ten       | Diem | Loai \n";
    cout<<" \n   ---- | ----- | ---- | ---- \n";
    for(i=1;i<=n;i++)
    {
        cout<<left;
        cout<<"   "<<setw(6)<<sv[i].masv<<" | "<<setw(20)<<sv[i].hoten
        <<" | "<<setw(3)<<sv[i].tb<<" | "<<sv[i].ploai<<"\n";
    }
    cout<<" \n-----\n\n";
    system("pause");
    return 0;
}

```

Kết quả chạy chương trình :

Nhap so luong sinh vien n = 3

Nhap so lieu sinh vien thu 1 :

Nhap so mon hoc (m): 2

Nhap ma sinh vien : 12345

Nhap ho ten sinh vien : Nguyen Van Ha

Nhap ten lop : it3

Nhap diem mon thu 1 = 8

Nhap diem mon thu 2 = 9

Nhap so lieu sinh vien thu 2 :

Nhap so mon hoc (m): 2

Nhap ma sinh vien : 32456

Nhap ho ten sinh vien : Tran Hong Vuong

Nhap ten lop : it7

Nhap diem mon thu 1 = 5

Nhap diem mon thu 2 = 6

Nhap so lieu sinh vien thu 3 :

Nhap so mon hoc (m): 2

Nhap ma sinh vien : 78642

Nhap ho ten sinh vien : Ngo Van So

Nhap ten lop : it8

Nhap diem mon thu 1 = 3

Nhap diem mon thu 2 = 4

HIEN DANH SACH KET QUA

Ma sv :	Ho ten	Diem	Loai
12345	Nguyen Van Ha	8.5	A
32456	Tran Hong Vuong	5.5	C
78642	Ngo Van So	3.5	F